CENG113M Mock Final Exam

Rules

- You may use printed or handwritten reference materials (cheat sheets) without any page limitations.
- Provide your final answers strictly on the official answer sheet. Use the provided scratch paper for drafts and transfer your final work afterward. At the end of the exam, submit only the answer sheet.
- For your code, assume a perfect user. You do not need to handle input errors; assume the user will always enter valid values.

(Q1) (25 pts) Write the output of the following program.

```
def multiply(a, b=3):
    return a * b

def classify(n):
    if n % 2 == 0:
        return "even"
    return "odd"

print("1:", multiply(4))
print("2:", multiply(2, 5))
print("3:", classify(multiply(3)))

words = ["fig", "apple", "kiwi", "banana"]
words.sort()
print("4:", words)
print("5:", words[-1])

words.reverse()
removed = words.pop(1)
print("6:", words)
print("7:", removed)

sentence = "the quick brown fox"
parts = sentence.split(" ")
print("8:", len(parts))
print("9:", "-".join(parts[:2]))
print("10:", sentence.count("o"))

def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

for i in range(4):
```

```
35      print("11." + str(i) + ":", factorial(i))
36
37  def countdown(n):
38      if n == 0:
39          print("Done!")
40          return
41      print(n, classify(n))
42      countdown(n - 1)
43
44  countdown(5)
```

(Q2) (25 pts) A **perfect number** is a positive integer that equals the sum of its proper divisors (all divisors excluding itself). For example, $6 = 1+2+3$ and $28 = 1+2+4+7+14$ are both perfect numbers.

Write a program that reads a limit from the user and prints all perfect numbers up to that limit. You must use the following three functions:

- `get_divisors(n)` returns a list of all proper divisors of `n`
- `is_perfect(n)` returns `True` if `n` is a perfect number (must call `get_divisors`)
- `find_perfects(limit)` prints all perfect numbers up to `limit` (must call `is_perfect`)

Example output for `limit = 30`:

```
6
28
```

(Q3) (25 pts) Write the two recursive functions described below. Do **not** use loops in either function.

Part A — Write a recursive function `harmonic(n)` that computes the n-th partial sum of the harmonic series:

$$H(n) = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$$

```
print(harmonic(1))    # 1.0
print(harmonic(4))    # 2.0833...
```

Part B — Write a function `digital_root(n)` that repeatedly sums the digits of `n` until a single digit remains.

```
print(digital_root(9875))   # 9875 -> 29 -> 11 -> 2
print(digital_root(7))      # 7
```

Hint: The digit sum of `n` can be computed with a loop using `n % 10` to get the last digit and `n // 10` to remove it. Keep repeating until a single digit remains.

(Q4) (25 pts) Implement a `Vector2D` class that represents a 2-dimensional vector.

The class must have the following:

- `__init__(self, x, y)` stores two float coordinates `x` and `y`
- `add(self, other)` returns a **new** `Vector2D` whose components are the element-wise sum of the two vectors
- `magnitude(self)` returns $\sqrt{x^2 + y^2}$ (you may use `** 0.5`)
- `is_unit(self)` returns `True` if the magnitude equals `1.0`
- `display(self)` prints in the format `Vector(x, y)`

The first two lines of code are provided. Your implementation must produce correct results for any values of `x` and `y`:

```
v1 = Vector2D(3, 4)
v2 = Vector2D(1, 2)
```

Using `v1` and `v2`, demonstrate all methods in your answer.

End of Exam